

Universidade da Beira Interior

Departamento de Informática



**Departamento de
Informática**

Deteção e Mitigação de Anomalias em Redes IoT

Elaborado por:

Vasco Ribeiro Colaço

Orientador:

Professor Doutor Bruno Miguel Correia Silva

28 de junho de 2026

Agradecimentos

A conclusão deste trabalho, bem como da grande maioria da minha vida académica, não seria possível sem a ajuda e o apoio de todas as pessoas que, de forma direta ou indireta, contribuíram para o meu percurso, incentivando-me a prosseguir e a ultrapassar as dificuldades.

Em primeiro lugar, quero expressar a minha sincera gratidão ao meu orientador, Professor Doutor Bruno Miguel Correia Silva, pela disponibilidade, pelos conselhos técnicos e pela orientação ao longo de todo o desenvolvimento deste projeto. Também agradecer à pessoa que acompanhou o desenvolvimento do meu projeto, Doutor Rui Pedro Pinto.

Agradeço também à Universidade da Beira Interior e ao Departamento de Informática por proporcionarem as condições necessárias à realização deste trabalho.

Por último, e não menos importante, um agradecimento especial à minha família e amigos pelo apoio incondicional, pela paciência e pelo encorajamento ao longo de todo o meu percurso académico.

Vasco Ribeiro Colaço
Covilhã, junho de 2026

Conteúdo

Conteúdo	iii
Lista de Figuras	v
Lista de Tabelas	vii
1 Introdução	1
1.1 Enquadramento	1
1.2 Motivação	2
1.3 Objetivos	2
1.4 Organização do Documento	3
2 Estado da Arte	5
2.1 Introdução	5
2.2 Segurança em Redes IoT e a Evolução da Detecção de Intrusões	5
2.3 Desafios Arquiteturais: Análise de Fluxos e o Paradoxo da La- tência	6
2.4 Posicionamento do Projeto e Comparação com a Literatura . .	7
3 Tecnologias e Ferramentas Utilizadas	9
3.1 Introdução	9
3.2 Linguagens de Programação e Bibliotecas	9
3.3 Algoritmos de Machine Learning	10
3.3.1 XGBoost	10
3.3.2 Random Forest Classifier	10
3.3.3 LightGBM	11
3.3.4 MLP	11
3.4 Arquitetura (IoT)	11
3.4.1 Edge Computing	11
3.5 Conclusões	11
4 Software e Arquitetura do Sistema	13
4.1 Introdução	13
4.2 Levantamento de Requisitos	13

4.2.1	Requisitos Funcionais	13
4.2.2	Requisitos Não-Funcionais	14
4.3	Modelação Visual do Sistema (UML)	14
4.3.1	Diagrama de Atividade (Fluxograma)	15
4.3.2	Diagrama de Estado	15
4.3.3	Diagrama de Sequência	16
5	Implementação e Testes	19
5.1	Introdução	19
5.2	Pré-processamento e Engenharia de Dados	19
5.2.1	Categorização de Anomalias e Rotulagem	20
5.2.2	O Papel dos Label Encoders	20
5.2.3	Filtro de Pureza e Normalização	21
5.3	Metodologia de Treino e Persistência	21
5.4	Mecanismo de Filtragem	22
5.4.1	Implementação Algorítmica	23
5.5	Ambiente de Simulação em Tempo Real	24
5.6	Resultados e Otimização da Mitigação	24
5.6.1	Mecanismo de Filtragem Rápida via Blacklist Dinâmica	24
5.6.2	Métricas de Avaliação	25
5.6.3	Análise do Desempenho Global	26
5.6.4	Recall por Anomalia	27
5.7	Decisão Final e Conclusões	29
6	Conclusões e Trabalho Futuro	31
6.1	Conclusões Principais	31
6.2	Trabalho Futuro	32
	Bibliografia	33

Lista de Figuras

4.1	Fluxograma do ciclo de decisão e mitigação autónoma no nó <i>Edge</i> .	15
4.2	Diagrama de Estado do motor de segurança no Servidor <i>Edge</i>	16
4.3	Diagrama de Sequência ilustrando a interceção e o bloqueio de um ataque na periferia da rede.	17
5.1	Arquitetura montada no Mininet.	24
5.2	Métricas Globais no Teste Cego - Mundo Real.	26
5.3	Tempo Médio de Inferência por Modelo de IA.	26
5.4	Variação e Estabilidade da Latência (Boxplot).	27
5.5	Taxa de Acertos (Recall %) por Tipo de Anomalia.	28
5.6	Precisão Global de Detecção (Accuracy %) por modelo.	29

Lista de Tabelas

2.1	Comparação de <i>Trade-offs</i> : Literatura <i>vs.</i> Arquitetura Proposta. . .	8
-----	---	---

Acrónimos

DDoS	<i>Distributed Denial of Service</i>
IA	Inteligência Artificial
IoT	<i>Internet of Things</i>
IP	<i>Internet Protocol</i>
LightGBM	<i>Light Gradient Boosting Machine</i>
MLP	<i>Multi-Layer Perceptron</i>
RAM	<i>Random Access Memory</i>
TCP	<i>Transmission Control Protocol</i>
UBI	Universidade da Beira Interior
XGBoost	<i>Extreme Gradient Boosting</i>

Glossário

Edge Computing É uma framework distribuída que aproxima as aplicações corporativas das fontes de dados, como os dispositivos de IoT ou servidores de *edge* locais.

IoT (*Internet of Things*) Refere-se à rede de objetos físicos, como sensores, câmaras e atuadores, que contêm tecnologia incorporada para comunicar e interagir através da Internet.

Machine Learning Aprendizagem Automática. É um subcampo da Inteligência Artificial que utiliza algoritmos e modelos estatísticos para que os computadores aprendam a executar tarefas a partir de dados.

MQTT *Message Queuing Telemetry Transport*. É um protocolo de mensagens leve, baseado no modelo *Publisher-Subscriber*, concebido especificamente para dispositivos com recursos limitados e redes de baixa largura de banda.

Docker Plataforma de *software* que permite empacotar, distribuir e executar aplicações dentro de contentores isolados. Consome muito menos recursos (RAM e Disco) em comparação com Máquinas Virtuais tradicionais.

iptables Utilitário de linha de comandos do sistema operativo Linux que permite a um administrador de sistemas configurar as regras do *firewall* fornecidas pelo *kernel*, controlando o tráfego de rede de entrada e de saída.

Mirai Botnet É um *malware* notório que infeta sistemas baseados em Linux, especialmente dispositivos IoT antigos ou com credenciais de fábrica, transformando-os numa rede de *bots* (Zombies) para lançar ataques DDoS em larga escala.

Spoofing Técnica de falsificação onde um atacante se faz passar por outro dispositivo na rede (falsificando o endereço IP ou MAC) para obter acesso não autorizado ou desviar tráfego.

Zero-Day Refere-se a ameaças ou anomalias que o sistema de segurança ou o modelo de Inteligência Artificial nunca viu durante a sua fase de treino, testando a sua capacidade de generalização e deteção de ataques inéditos.

TCP/IP Conjunto de protocolos de comunicação fundamentais da Internet. Resulta da união entre o protocolo de controlo de transmissão (TCP) e o protocolo de inter-rede (IP).

Firewall Dispositivo físico ou *software* (como o *iptables*) desenvolvido para aplicar políticas de segurança, bloqueando ou permitindo o tráfego de dados com base num conjunto de regras predefinidas.

Capítulo

1

Introdução

1.1 Enquadramento

Na última década, o conceito de *Smart Cities* (Cidades Inteligentes) tem-se expandido rapidamente, apoiado pelo aumento de dispositivos *Internet of Things* (IoT). Estes dispositivos, que incluem desde sensores de temperatura e semáforos inteligentes a câmaras de vigilância, recolhem e transmitem dados continuamente para melhorar a eficiência urbana e a qualidade de vida dos cidadãos.

No entanto, a adoção massiva destas tecnologias trouxe desafios de segurança. Os dispositivos IoT possuem, na sua esmagadora maioria, capacidades computacionais e de armazenamento extremamente limitadas, o que impossibilita a execução de ferramentas de antivírus ou *firewalls* complexas. Esta vulnerabilidade tem sido explorada por cibercriminosos para lançar ataques devastadores, como o software malicioso *Mirai*, que transforma milhares de dispositivos vulneráveis numa rede *zombie* capaz de orquestrar ataques de *Distributed Denial of Service* (DDoS) à escala global.

É neste contexto que surge o paradigma de *Edge Computing*, que propõe a deslocação do poder de processamento da nuvem (*Cloud*) para a periferia da rede (o *Edge*), como por exemplo os *routers* de acesso. Ao colocar a inteligência mais próxima da origem dos dados, torna-se possível analisar o tráfego e intercepar ameaças em tempo real.

Este projeto foi desenvolvido no contexto da unidade curricular de Projeto da Universidade da Beira Interior (UBI), visando a criação de uma solução prática e inovadora para este desafio.

1.2 Motivação

A motivação principal para o desenvolvimento deste projeto prende-se com a ineficiência das soluções de segurança tradicionais quando aplicadas a redes IoT. A abordagem clássica de enviar todo o tráfego de rede para servidores centrais na nuvem para análise de segurança gera uma latência insustentável e consome uma largura de banda massiva. Em cenários críticos de uma *Smart City*, um atraso de poucos segundos na deteção de um ataque pode resultar na indisponibilidade de serviços essenciais.

Além disso, as regras estáticas de bloqueio de *Internet Protocol* (IP) são insuficientes contra ameaças modernas e evasivas (como *Spoofing* ou *Zero-Day*). É então fundamental colocar sistemas autônomos de Inteligência Artificial (IA) na periferia da rede, capazes de aprender os padrões legítimos e intercepar anomalias de forma cirúrgica, minimizando a sobrecarga no *hardware*.

1.3 Objetivos

O objetivo central deste projeto é desenvolver, testar e emular um sistema autônomo de deteção e mitigação de ameaças cibernéticas otimizado para *Edge Computing* em redes IoT.

Para atingir este propósito, definiram-se os seguintes objetivos específicos:

1. Estudar e pré-processar um conjunto de dados moderno e representativo de tráfego IoT (o *dataset* CICIoT2023), aplicando técnicas de balanceamento de classes para resolver a assimetria entre tráfego normal e anômalo.
2. Treinar e comparar o desempenho de diferentes algoritmos de *Machine Learning*, focando a análise no equilíbrio entre a taxa de acertos e o consumo de recursos computacionais (*Light Gradient Boosting Machine* (LightGBM), *Extreme Gradient Boosting* (XGBoost), Random Forest e *Multi-Layer Perceptron* (MLP)).
3. Desenvolver um módulo de mitigação automatizada que atue como ponte entre as previsões do modelo de IA e a componente de *firewall* do sistema operativo (*iptables*), aplicando regras de bloqueio ou quarentena em tempo real.
4. Validar o sistema desenvolvido num ambiente realista de rede, recorrendo ao emulador *Mininet* para construir uma topologia representa-

tiva de *Edge Computing*, comprovando a eficácia da solução proposta na interceção e mitigação de anomalias em tempo real.

1.4 Organização do Documento

De modo a refletir o trabalho desenvolvido e a metodologia aplicada, este documento encontra-se estruturado em seis capítulos principais:

- O **Capítulo 1 – Introdução** – enquadra o tema das redes IoT e da segurança cibernética, descrevendo a motivação, o problema a resolver e os objetivos propostos para o projeto.
- O **Capítulo 2 – Estado da Arte** – apresenta uma revisão bibliográfica sobre as abordagens atuais de deteção de intrusões em IoT, evidenciando as lacunas existentes e justificando a necessidade de abordagens baseadas em *Machine Learning* no *Edge*.
- O **Capítulo 3 – Tecnologias e Ferramentas Utilizadas** – descreve os conceitos teóricos fundamentais, os algoritmos de IA selecionados e as ferramentas de emulação de rede que suportam o desenvolvimento da arquitetura.
- O **Capítulo 4 – Software e Arquitetura do Sistema** – descreve o levantamento de requisitos, a arquitetura modular da aplicação e o fluxo de execução do sistema de mitigação autónoma.
- O **Capítulo 5 – Implementação e Testes** – detalha todo o processo de engenharia de dados, o treino dos modelos, a análise crítica dos resultados do "Teste Cego" e a justificação técnica para a escolha final do algoritmo.
- O **Capítulo 6 – Conclusões e Trabalho Futuro** – sintetiza as principais descobertas do projeto, reflete sobre o cumprimento dos objetivos iniciais e propõe futuras linhas de investigação e melhorias do sistema desenvolvido.

Capítulo

2

Estado da Arte

2.1 Introdução

A expansão de dispositivos da *Internet of Things* (IoT) trouxe consigo desafios de segurança sem precedentes. Este capítulo apresenta uma revisão da literatura sobre as vulnerabilidades inerentes às redes IoT, a evolução da cibersegurança suportada por *Machine Learning* (ML) e os constrangimentos arquiteturais de implementar estas soluções na periferia da rede (*Edge Computing*).

2.2 Segurança em Redes IoT e a Evolução da Detecção de Intrusões

Os dispositivos IoT, como sensores e atuadores, são frequentemente desenhados com restrições ao nível do processamento, memória e consumo energético [1]. Esta limitação computacional inviabiliza a instalação de protocolos de segurança complexos ou soluções de antivírus tradicionais nos próprios equipamentos. Consequentemente, a superfície de ataque nestas redes é vasta, tornando-as alvos preferenciais para o recrutamento em *Botnets* e para a orquestração de ataques de Negação de Serviço Distribuída (DDoS) [2]. Adicionalmente, a heterogeneidade dos protocolos de comunicação dificulta a padronização das defesas, exigindo mecanismos que operem diretamente na camada de rede [3].

Historicamente, os Sistemas de Detecção de Intrusão (IDS) basearam-se em catálogos de assinaturas (*Signature-based IDS*), comparando o tráfego com padrões conhecidos de *malware*. Embora eficazes contra ameaças documen-

tadas, estes sistemas revelam-se ineficazes perante ataques *Zero-Day* ou mutações de código malicioso [4]. Para colmatar esta lacuna, a comunidade científica tem transitado para abordagens baseadas em anomalias suportadas por *Machine Learning*. Modelos como *Random Forest* [5], *Support Vector Machines* (SVM) e Redes Neurais Artificiais (ANN) demonstraram uma elevada capacidade para modelar o comportamento "normal" da infraestrutura e classificar desvios estatísticos como potenciais intrusões [6].

2.3 Desafios Arquiteturais: Análise de Fluxos e o Paradoxo da Latência

A eficácia de um modelo de Inteligência Artificial depende diretamente do método de extração de características (*features*) do tráfego. A Inspeção Profunda de Pacotes (*Deep Packet Inspection* - DPI) analisa o conteúdo integral da mensagem (*payload*), garantindo alta precisão. Contudo, o DPI levanta sérias questões de privacidade e exige um esforço computacional incompatível com *routers* de baixo custo [3]. Em alternativa, a análise baseada em fluxos (*Flow-based Analysis*) extrai apenas metadados estatísticos da comunicação, tais como o tempo entre pacotes, a duração do fluxo e a contagem de *flags* TCP. Ferramentas como o *CICFlowMeter*, utilizadas para a geração de *datasets* como o *CICIoT2023* [7], provam que esta abordagem garante a preservação da privacidade e reduz drasticamente a sobrecarga de processamento.

Esta redução de carga computacional é um fator crítico quando se introduz o paradigma de *Edge Computing*, que propõe a deslocação do processamento de dados da nuvem (*Cloud*) para a periferia da rede [8]. No entanto, a literatura atual evidencia um paradoxo na aplicação destas tecnologias: a esmagadora maioria dos estudos foca-se exclusivamente na maximização da exatidão (*Accuracy*) preditiva, recorrendo a *Ensembles* de árvores de decisão altamente complexos. O problema fundamental desta abordagem reside no negligenciar do "orçamento de latência". Em cenários de ataques volumétricos, o tempo que um *Edge Router* demora a processar a inferência matemática de uma árvore profunda excede o tempo de chegada do pacote seguinte, resultando no bloqueio do *buffer* e na negação de serviço provocada pelo próprio sistema de defesa.

2.4 Posicionamento do Projeto e Comparação com a Literatura

Em contraste com grande parte da literatura existente, o presente trabalho não procura unicamente a precisão estatística isolada, mas sim a resolução do problema da latência em ambientes de *Edge Computing*.

Para estabelecer um termo de comparação direto e fundamentar as decisões arquiteturais tomadas, é pertinente analisar trabalhos recentes com metodologias de simulação semelhantes. Destaca-se o estudo focado em *SDN-Enabled IoT Anomaly Detection Using Ensemble Learning* [9], que também recorre ao emulador Mininet para validar mecanismos de segurança numa rede IoT. Nesse estudo, os autores propõem a utilização de *Ensembles* de aprendizagem profunda (combinando múltiplas redes e árvores) colocados num controlador central para maximizar a exatidão e o *F1-Score*. Embora a abordagem alcance uma precisão teórica notável na deteção de ataques dinâmicos, os autores focam-se quase exclusivamente no poder preditivo abstrato, não acautelando a sobrecarga computacional de processar modelos tão complexos em tempo real.

É exatamente nesta lacuna que este projeto se demarca. A Tabela 2.1 sintetiza os compromissos assumidos neste trabalho, em oposição à abordagem tradicional focada na maximização do acerto.

Este trabalho assume que, em cenários de ataques volumétricos (como injeções massivas de pacotes UDP/TCP), utilizar *Ensembles* pesados na periferia da rede resulta numa latência insustentável. Ao avaliar empiricamente o custo-benefício, o presente projeto coroa modelos mais leves, como o MLP, sacrificando uma margem negligenciável de exatidão pura em prol da sobrevivência da rede.

Adicionalmente, a inovação prática deste projeto reside na hibridização do sistema: ao acoplar o modelo MLP a uma arquitetura de *Blacklists* dinâmicas operada diretamente no Servidor *Edge* (atuando como um nó de segurança dedicado), garante-se uma latência de processamento contínua mínima. Esta separação física e lógica de funções liberta o *router* principal da pesada carga de filtragem, impedindo o bloqueio do seu *buffer* e viabilizando a proteção em tempo real para infraestruturas com recursos limitados.

Tabela 2.1: Comparação de *Trade-offs*: Literatura *vs.* Arquitetura Proposta.

Critério de Avaliação	Artigo de Referência [9]	O Nosso Projeto (MLP + <i>Blacklist</i>)
Topologia de Rede	Controlador Central SDN	Nó de <i>Edge Computing</i> autônomo
Algoritmo de IA	<i>Ensemble Learning</i> (Múltiplos modelos pesados)	<i>Multi-Layer Perceptron</i> (Cálculo matricial rápido)
Foco Principal	Maximização absoluta da Exatidão (<i>Accuracy</i>)	Equilíbrio ótimo entre Exatidão preditiva e Latência
Mecanismo de Mitigação	O controlador SDN processa e bloqueia de forma contínua	Arquitetura Híbrida (IA analisa o 1º pacote; <i>Blacklist</i> bloqueia os restantes)
Latência Operacional	Alta (inviável para processamento de pacotes individuais)	Ultrabaixa (~1 ms por inferência preditiva)
Risco em DDoS	Risco severo de exaustão de recursos e estrangulamento da rede	Mitigação quase instintiva, poupando a capacidade do <i>Buffer</i>

Capítulo

3

Tecnologias e Ferramentas Utilizadas

3.1 Introdução

O desenvolvimento de um sistema capaz de realizar a detecção e mitigação de anomalias no tráfego de uma *Smart City* exige uma seleção rigorosa de tecnologias que equilibrem o poder preditivo com a eficiência computacional. Dada a natureza da infraestrutura em dispositivos IoT, em que é necessário processar fluxos massivos de dados em tempo real, frequentemente com recursos de hardware limitados.

Este capítulo detalha as linguagens, bibliotecas e os fundamentos teóricos dos algoritmos de *Machine Learning* escolhidos para este projeto. O capítulo encontra-se organizado da seguinte forma: a Secção 3.2 apresenta o ecossistema de programação; a Secção 3.3 explora os quatro algoritmos avaliados (XGBoost, Random Forest, LightGBM e MLP); a Secção 3.4 descreve o paradigma de *Edge Computing*; finalmente, a Secção 3.5 sintetiza as conclusões do capítulo.

3.2 Linguagens de Programação e Bibliotecas

A implementação do fluxo de dados e o treino dos modelos foram desenvolvidos em **Python**. Esta escolha justifica-se pela vasta disponibilidade de bibliotecas especializadas em *Data Science* e Cibersegurança. Destacam-se as seguintes ferramentas:

- **Pandas:** Utilizada para a manipulação de *DataFrames*. Foi essencial

para fundir os volumes do *dataset* CICIoT2023 e realizar a amostragem estratificada para equilibrar as classes [10].

- **Scikit-Learn:** Utilizada para a divisão de dados (`train_test_split`), implementação do `LabelEncoder` e cálculo de métricas como o F1-Score [11]. O `LabelEncoder` é vital para traduzir categorias textuais (ex: "DDoS-TCP") em formato numérico processável pelos modelos.
- **XGBoost e LightGBM:** Bibliotecas de *Gradient Boosting* otimizadas. Enquanto o XGBoost foca-se na precisão máxima [12], o LightGBM é desenhado para ser "leve" e rápido, consumindo menos memória *Random Access Memory* (RAM), sendo ideal para cenários de *Edge Computing* [13].
- **Joblib:** Crucial para a persistência dos modelos e do `LabelEncoder` em ficheiros *.pkl*. Estes ficheiros permitem que o modelo treinado seja carregado instantaneamente no *Edge Router* sem necessidade de re-treino.
- **Mininet e Emulação de Redes Virtuais:** Um emulador de redes de computadores capaz de criar uma rede virtual completa — contendo *switches*, *routers* e *hosts*.

3.3 Algoritmos de Machine Learning

Neste projeto, procedeu-se a uma análise comparativa entre algoritmos baseados em árvores de decisão e redes neurais [4, 14].

3.3.1 XGBoost

O XGBoost foi selecionado como o "vencedor técnico" preliminar devido à sua elevada precisão [12, 15]. Ele utiliza uma arquitetura de *Gradient Boosting* que permite capturar padrões não-lineares complexos em tráfego de rede, distinguindo com eficácia nuances entre diferentes tipos de ataques TCP e UDP [6].

3.3.2 Random Forest Classifier

Ao contrário do regressor inicial, utilizou-se a versão de classificação para identificar categorias de ataques. O *Random Forest* constrói múltiplas árvores de decisão independentes, sendo particularmente robusto face a dados ruidosos. Revelou-se essencial para detetar ataques furtivos como *Recon* e *Spoofing*.

3.3.3 LightGBM

O LightGBM destaca-se pela sua eficiência em *Edge Computing* [13]. Ao crescer por folhas (*leaf-wise*) em vez de níveis, reduz a latência de inferência e o tamanho do ficheiro final, sendo ideal para dispositivos com armazenamento limitado [4].

3.3.4 MLP

Para divergir da lógica de árvores, introduziu-se o MLP, uma rede neuronal que utiliza "neurónios" matemáticos interligados [6]. Esta arquitetura tenta mimetizar o cérebro humano para detetar padrões subtis que regras rígidas de "se/então" podem ignorar [4].

3.4 Arquitetura (IoT)

3.4.1 Edge Computing

O paradigma de *Edge Computing* aproxima a inteligência artificial da fonte dos dados. Ao executar os modelos diretamente nos *Routers* da *Smart City*, a mitigação de ataques é realizada com latência mínima, descartando pacotes maliciosos antes que estes atinjam o núcleo da rede [8].

Em termos de hardware, os nós *Edge* típicos dispõem de processadores de baixo consumo (ARM ou x86 de gama de entrada), memória *Random Access Memory* (RAM) limitada (geralmente entre 512 MB e 2 GB) e armazenamento reduzido. Esta escassez de recursos impõe restrições estritas ao tamanho dos modelos de IA e ao seu tempo de inferência, tornando o equilíbrio entre exatidão preditiva e eficiência computacional o critério de seleção dominante, conforme analisado na Secção 3.5.

3.5 Conclusões

A análise tecnológica efetuada neste capítulo permitiu estabelecer uma base sólida para a seleção dos algoritmos a avaliar empiricamente. Os modelos baseados em árvores de decisão, nomeadamente o XGBoost e o Random Forest, destacam-se pela sua elevada capacidade preditiva, sendo candidatos naturais para cenários onde a precisão estatística é o critério dominante.

O LightGBM surge como um candidato intermédio promissor para *Edge Computing*, combinando uma boa precisão com menor consumo de memória e tempos de inferência potencialmente mais reduzidos face aos modelos

anteriores. Contudo, esta hipótese carece de validação empírica: apenas a avaliação comparativa em ambiente de simulação, apresentada no Capítulo 5, permitirá confirmar se os modelos baseados em árvores conseguem satisfazer os requisitos críticos de latência e estabilidade impostos por um *Edge Router* sob ataque volumétrico.

É neste contexto que o MLP, pela sua arquitetura matricial intrínseca, se apresenta como candidato promissor para *deployment* na periferia da rede. A sua natureza de cálculo matricial sugere potencialmente uma velocidade operacional e estabilidade determinística superiores às abordagens baseadas em árvores. Contudo, apenas a avaliação empírica no Capítulo 5 permitirá confirmar se o MLP consegue satisfazer os requisitos críticos de latência e em que medida sacrifica exatidão preditiva para o efeito. Estas ferramentas formam, em conjunto, a base para a implementação e análise crítica detalhadas no capítulo seguinte.

Software e Arquitetura do Sistema

4.1 Introdução

O desenvolvimento de um Sistema de Prevenção de Intrusões (IPS) assente em Inteligência Artificial para operar na periferia da rede (*Edge*) exige um planeamento arquitetural rigoroso. Este capítulo detalha o processo de Engenharia de Software subjacente ao projeto, garantindo que a transição dos modelos matemáticos para um ambiente de execução real respeita os princípios de modularidade, eficiência e tolerância a falhas. Serão abordados o levantamento de requisitos, a separação de componentes lógicos e o fluxo de execução do sistema.

4.2 Levantamento de Requisitos

A especificação de requisitos define o contrato técnico que o software deve cumprir para solucionar o problema proposto de forma eficaz num ambiente de *Smart City*.

4.2.1 Requisitos Funcionais

Os Requisitos Funcionais (RF) descrevem as ações e comportamentos específicos que o sistema tem de obrigatoriamente executar:

- **RF01 - Interceção de Tráfego:** O sistema deve ser capaz de capturar e inspecionar pacotes de rede (TCP, UDP e ICMP) que atravessem o *Edge Router*.

- **RF02 - Classificação de Ameaças:** O sistema deve utilizar um modelo preditivo para classificar os fluxos de rede, distinguindo tráfego legítimo de classes de anomalias (incluindo *DDoS*, *Mirai* e *BruteForce*).
- **RF03 - Mitigação Autônoma:** Ao detetar uma anomalia, o sistema deve comunicar dinamicamente com o *firewall* do sistema operativo (*iptables*) para injetar regras cirúrgicas de bloqueio (*DROP* ou *REJECT*).
- **RF04 - Auditoria:** Todas as intervenções ativas do sistema devem gerar um registo local (*log*) contendo o IP atacante, a classe da anomalia e o tempo computacional gasto na inferência.

4.2.2 Requisitos Não-Funcionais

Os Requisitos Não-Funcionais (RNF) definem os critérios de qualidade e as restrições físicas impostas pela infraestrutura IoT:

- **RNF01 - Latência Ultrabaixa:** O processo completo, desde a extração das *features* até à decisão de bloqueio, não deve exceder um orçamento de tempo de resposta na ordem dos ~ 2 milissegundos por pacote.
- **RNF02 - Eficiência de Memória:** O modelo de inferência deve ser estático e otimizado em tamanho, minimizando a pegada (*footprint*) na memória RAM do hardware *Edge*.
- **RNF03 - Autonomia Operacional:** O sistema deve operar de forma inteiramente isolada, sem qualquer dependência de consultas a servidores na nuvem (*Cloud*) para a tomada de decisões, prevenindo pontos únicos de falha e latência induzida pela rede externa.

4.3 Modelação Visual do Sistema (UML)

Para garantir que a arquitetura do software é compreendida de forma clara e que a sua implementação respeita os rigorosos requisitos de *Edge Computing*, o comportamento do sistema foi modelado a partir de três perspetivas complementares. Esta modelação visual detalha a lógica de decisão algorítmica, as transições de estado do servidor e a sequência cronológica de comunicação entre os diversos componentes da rede.

4.3.1 Diagrama de Atividade (Fluxograma)

O Diagrama de Atividade ilustra a rotina de processamento a que cada novo fluxo de rede é submetido ao chegar ao nó *Edge*. O foco deste diagrama é a ramificação lógica (*branching*) que ocorre após a fase de inferência matemática. Conforme ilustrado na Figura 4.1, o sistema atua como uma bifurcação estrita: se a classificação indicar tráfego benigno, o pacote prossegue sem interferência; se indicar uma ameaça, o fluxo é desviado para o módulo de injeção de regras, culminando no descarte sumário do pacote.

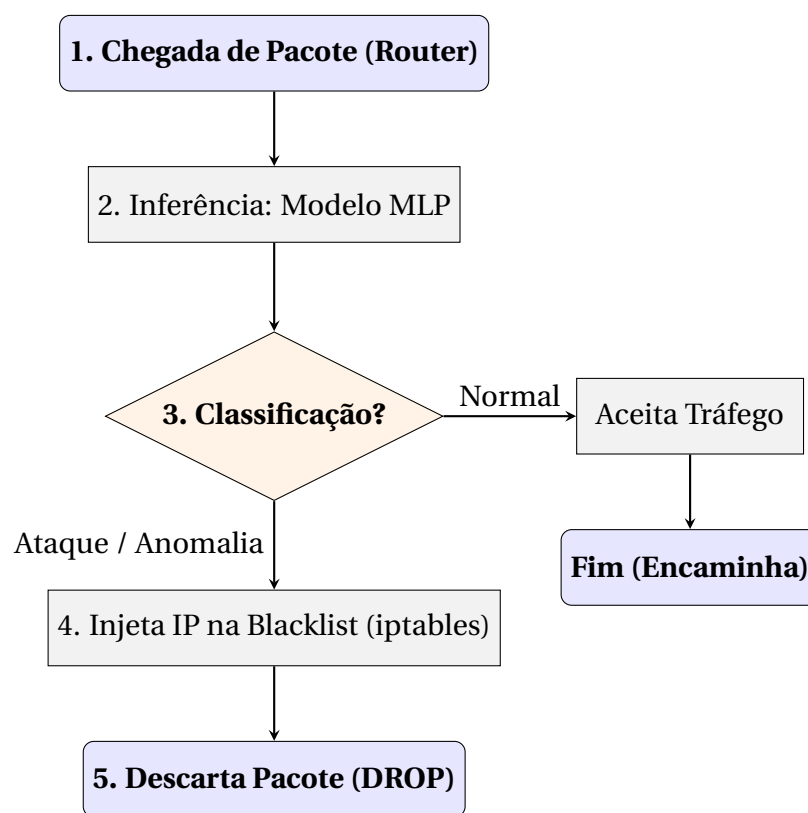


Figura 4.1: Fluxograma do ciclo de decisão e mitigação autónoma no nó *Edge*.

4.3.2 Diagrama de Estado

Enquanto o fluxograma foca-se nos dados, o Diagrama de Estado (Figura 4.2) descreve os modos operacionais em que o próprio servidor *Edge* se pode encontrar num dado momento. O motor de segurança foi concebido para ser altamente reativo, mantendo-se num estado de *Escuta (Idle)* com consumo mínimo de recursos. A transição para o estado de *Inferência* é engatilhada

apenas pela interceção de um pacote. A passagem ao estado crítico de *Mitigação Ativa* é reservada estritamente para cenários de ataque, após os quais o sistema retoma imediatamente a sua postura de monitorização passiva.

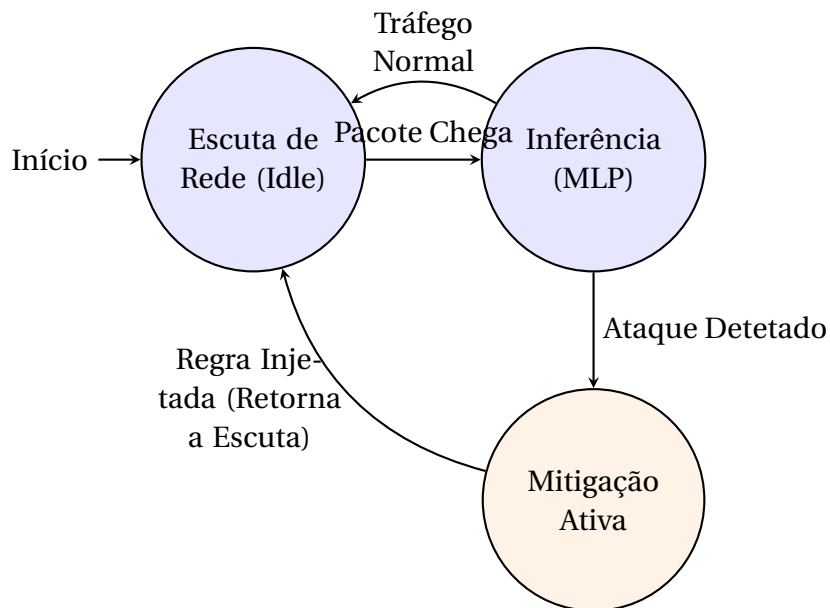


Figura 4.2: Diagrama de Estado do motor de segurança no Servidor *Edge*.

4.3.3 Diagrama de Sequência

O Diagrama de Sequência (Figura 4.3) é o elemento visual mais importante para compreender o ganho de desempenho da hibridização do sistema (Inteligência Artificial suportada por *Blacklists*). Ele estabelece a linha cronológica de comunicação entre os atores da rede.

É possível observar que apenas o primeiro pacote enviado pelo atacante obriga ao processamento analítico profundo no nó *Edge*. Assim que a regra de mitigação é injetada no *firewall* (*iptables*), qualquer tentativa subsequente do mesmo atacante é barrada diretamente ao nível do *Kernel*, poupando assim os recursos da CPU e prevenindo a exaustão do sistema de defesa.

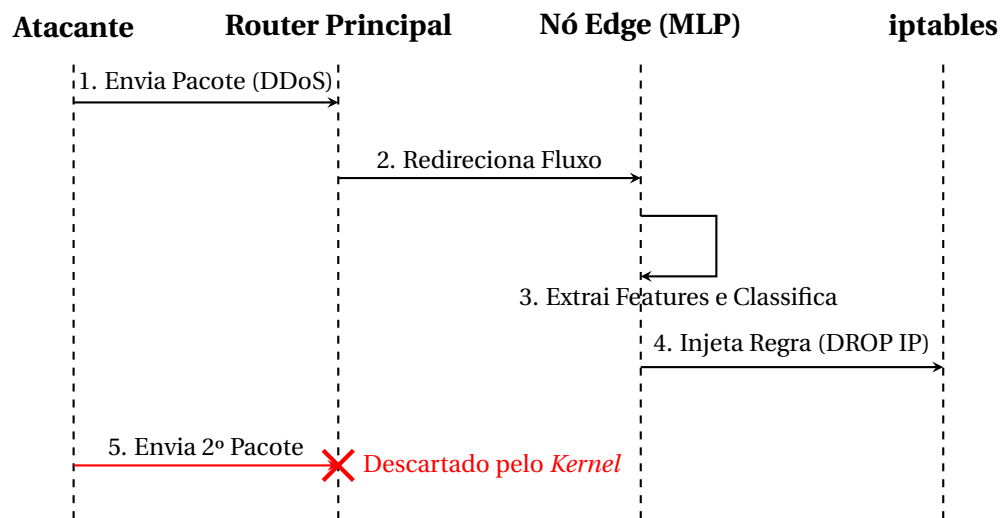


Figura 4.3: Diagrama de Sequência ilustrando a intercepção e o bloqueio de um ataque na periferia da rede.

Capítulo

5

Implementação e Testes

5.1 Introdução

Após a fundamentação teórica, este capítulo descreve a materialização prática do sistema de detecção de anomalias para redes IoT. O foco incide sobre a criação de um *dataset* equilibrado, o tratamento de discrepâncias estatísticas e a avaliação de quatro modelos de Inteligência Artificial: XGBoost, LightGBM, Random Forest e MLP. O capítulo encontra-se organizado da seguinte forma: a Secção 5.2 detalha o pré-processamento de dados; a Secção 5.3 descreve a metodologia de treino e persistência; a Secção 5.4 apresenta o mecanismo de filtragem ativa e mitigação automática de ameaças; a Secção 5.5 expõe o ambiente de simulação em tempo real; a Secção 5.6 elabora a análise rigorosa das métricas em ambiente *Zero-Day*; e, por fim, a Secção 5.7 apresenta a decisão final.

5.2 Pré-processamento e Engenharia de Dados

A eficácia da detecção depende da qualidade dos dados. Utilizou-se o dataset CIIoT2023, consolidando os volumes Merged01 a Merged05 para garantir uma base de amostragem rica para treino dos modelos e Merged06 a Merged10 para validação dos modelos treinados.

Numa primeira tentativa, verificou-se que treinar a Inteligência Artificial exclusivamente com assinaturas de anomalias induzia um viés (bias) extremo no modelo, resultando num fenómeno de over-triggering (taxa de Falsos Positivos próxima de 100% face a utilizadores legítimos). Para mitigar este problema, procedeu-se à extração de um dataset de treino equilibrado com mais

de 90.000 amostras, garantindo a integração de 10.000 instâncias de tráfego benigno (Normal).

5.2.1 Categorização de Anomalias e Rotulagem

Para que o sistema de monitorização seja capaz de atuar de forma precisa, a Inteligência Artificial foi treinada para identificar oito categorias distintas de anomalias:

- **Ataques Volumétricos TCP** (*DDoS-TCP* e *DoS-TCP*): inundação maliciosa de pedidos de sincronização TCP, mitigada através da injeção de regras iptables com ação DROP sobre o protocolo TCP do IP atacante;
- **Ataques Volumétricos UDP/ICMP** (*DDoS-UDP/ICMP* e *DoS-UDP/ICMP*): estrangulamento de largura de banda por inundação de datagramas e pacotes de controlo, mitigado com bloqueio simultâneo de tráfego UDP e ICMP ;
- **Ameaças de IoT Múltiplo** (*Mirai Botnet*): comprometimento total do dispositivo de origem por malware de botnet, mitigado com isolamento completo do IP atacante sem filtragem de protocolo (DROP absoluto);
- **Quebra de Senha** (*BruteForce*): tentativas automatizadas de acesso por força bruta, tipicamente dirigidas à porta 22 (SSH), mitigadas com REJECT focado no protocolo TCP e nessa porta específica;
- **Reconhecimento** (*Recon*): varrimento furtivo da rede para mapeamento de dispositivos e serviços ativos, detetado através de padrões estatísticos de fluxo;
- **Falsificação** (*Spoofing*): falsificação de endereços IP ou MAC para obtenção de acesso não autorizado, mitigada com isolamento completo do IP de origem (DROP absoluto).

A ação de mitigação específica aplicada a cada classe é executada autonomamente pelo módulo de filtragem ativa, detalhado na Secção 5.4.

5.2.2 O Papel dos Label Encoders

Os algoritmos de *Machine Learning* são baseados em equações matemáticas que operam exclusivamente com números. Por este motivo, é impossível entregar ao modelo categorias em formato de texto como "DDoS-TCP". O `LabelEncoder` atua como um dicionário de tradução bidirecional, mapeando

cada ataque único para um índice numérico (ex: "DDoS-TCP" → 2). Este tradutor é guardado num ficheiro *.pkl* para que, em ambiente de produção, o Router possa converter o número previsto pela IA de volta para um nome legível por humanos e sistemas de segurança.

5.2.3 Filtro de Pureza e Normalização

Identificou-se a presença de valores infinitos (`np.inf`) em métricas de volumetria, resultantes de anomalias na captura. Implementou-se um filtro que substituiu estes valores por zero, garantindo que algoritmos como o Random Forest não falhassem na compilação geométrica das matrizes. Para o MLP, aplicou-se o `StandardScaler` para normalizar a escala dos dados, evitando que variáveis com grandes magnitudes dominassem injustamente o treino.

5.3 Metodologia de Treino e Persistência

Os modelos foram treinados utilizando o "Dataset de Treino"equilibrado. O Excerto de Código 5.1 ilustra a rotina de treino e exportação do modelo LightGBM.

```
import pandas as pd
import lightgbm as lgb
import joblib

# 1. Carregar Dataset equilibrado
df = pd.read_csv('dataset_treino.csv')

# 2. Preparar X e y
y = df['label_encoded']
X = df.drop(['Label', 'label_encoded', 'categoria'], axis=1)

# 3. Treino do modelo focado em Edge
modelo = lgb.LGBMClassifier(n_estimators=100, random_state=42)
modelo.fit(X, y)

# 4. Guardar cerebro para o Router
joblib.dump(modelo, 'modelo_ciberseguranca_lgbm.pkl')
```

Excerto de Código 5.1: Rotina de treino e persistência do modelo LightGBM.

O padrão de treino e persistência apresentado é análogo para todos os modelos avaliados. No caso do MLP, o modelo é instanciado via `sklearn.neural_network.MLPClassifier` e os dados são previamente normalizados com `StandardScaler`, cujo estado é igualmente guardado em ficheiro *.pkl* para garantir que a normalização em produção é idêntica à aplicada durante o treino.

5.4 Mecanismo de Filtragem

A detecção atempada de uma anomalia através de algoritmos de *Machine Learning* constitui apenas a primeira fase de Sistema de Prevenção de Intrusões (IPS). Para que a infraestrutura de *Edge Computing* garanta a resiliência da *Smart City*, é imperativo que a inferência preditiva seja traduzida, em milissegundos, numa ação de bloqueio ao nível da rede. Este capítulo detalha a arquitetura de mitigação ativa desenvolvida, assente na manipulação dinâmica das tabelas de encaminhamento do *kernel* do sistema operativo através da ferramenta *iptables*.

Em vez de depender de soluções de *Firewall* externas de elevado custo, o nó *Edge* foi programado para atuar autonomamente, convertendo a saída do modelo numa regra de bloqueio estrita. A função de mitigação foi estruturada para ser granular, aplicando o princípio do menor privilégio: a penalização aplicada ao endereço de IP atacante é talhada à medida da classe de anomalia detetada, garantindo máxima eficácia com o mínimo impacto colateral.

A lógica de atuação do sistema obedece às seguintes regras de contenção:

- **Ataques Volumétricos TCP (DDoS-TCP e DoS-TCP):** Quando a rede é inundada por pedidos de sincronização maliciosos, o sistema invoca uma regra *iptables* que descarta silenciosamente (DROP) pacotes cujo protocolo seja estritamente TCP provenientes daquele IP. O uso da ação DROP (em vez de REJECT) é vital em cenários volumétricos, pois evita que o *router* desperdice poder computacional a enviar mensagens de "conexão recusada" de volta ao atacante.
- **Ataques Volumétricos UDP/ICMP (DDoS-UDP/ICMP e DoS-UDP/ICMP):** De forma análoga aos ataques TCP, a identificação destas classes gera duas regras em simultâneo: o bloqueio transversal de tráfego de datagramas (UDP) e o bloqueio de tráfego de controlo e testes de eco (ICMP), anulando táticas comuns de estrangulamento de largura de banda, como os *Ping Floods*.
- **Ameaças de IoT Múltiplo e Falsificação (Mirai e Spoofing):** Ataques orquestrados por *Botnets* como o *Mirai* representam um comprometimento total do dispositivo de origem. Como tal, o sistema aplica uma regra absolutista (DROP sem filtragem de protocolo), isolando por completo o IP atacante da rede, impedindo que este comunique com o exterior ou tente propagar o *malware* internamente na *Smart City*.
- **Quebra de Senha (BruteForce):** Ao detetar uma tentativa de força bruta (normalmente direcionada a serviços de administração remota), o sis-

tema foca o bloqueio especificamente no protocolo *Transmission Control Protocol* (TCP) e na porta 22 (serviço SSH). Adicionalmente, utiliza-se a ação REJECT. Esta resposta envia ativamente um pacote de recusa ao atacante, quebrando imediatamente a ferramenta automatizada de tentativa de *passwords*, fechando a porta sem interromper outro possível tráfego legítimo.

5.4.1 Implementação Algorítmica

O Excerto de Código 5.2 ilustra o *script* Python encarregue de orquestrar a *Blacklist* dinâmica. A função `aplicar_mitigacao` atua como uma ponte entre a camada lógica (Inteligência Artificial) e a camada física (Camada de Rede).

```
def aplicar_mitigacao(classe, ip_atacante, tempo_ms):
    if classe in ['DDoS-TCP', 'DoS-TCP']:
        executar_comando(f"iptables -A INPUT -p tcp -s {ip_atacante} -j DROP")

    elif classe in ['DDoS-UDP/ICMP', 'DoS-UDP/ICMP']:
        executar_comando(f"iptables -A INPUT -p udp -s {ip_atacante} -j DROP")
        executar_comando(f"iptables -A INPUT -p icmp -s {ip_atacante} -j DROP")

    elif classe == 'Mirai-Botnet' or classe == 'Spoofing':
        executar_comando(f"iptables -A INPUT -s {ip_atacante} -j DROP")

    elif classe == 'BruteForce':
        executar_comando(f"iptables -A INPUT -p tcp --dport 22 -s {ip_atacante} -j REJECT")

    # Auditoria e Registro de Desempenho
    print(f"-->Mitigacao aplicada para: {classe} (IP: {ip_atacante}) - Tempo de IA: {tempo_ms:.2f} ms")
```

Excerto de Código 5.2: Lógica de Mitigação Ativa baseada em iptables.

Por fim, cada intervenção do *Edge Router* gera um evento de auditoria no *log* do sistema (incluindo o IP isolado, o tipo de ameaça e o custo de inferência algorítmica). Estes registros comprovam que o ciclo completo de defesa — desde a leitura do pacote, inferência pela rede neuronal e injeção da regra de bloqueio — decorre nos exigentes limites impostos pelo paradigma de operações em tempo real.

5.5 Ambiente de Simulação em Tempo Real

Para validar a viabilidade dos modelos de *Machine Learning* num cenário realista de *Edge Computing*, construiu-se uma topologia de rede virtualizada com recurso à ferramenta Mininet. De forma a refletir a complexidade de uma infraestrutura autêntica, a arquitetura foi composta por três elementos fundamentais: um nó atacante (h1), responsável pela injeção em massa dos pacotes provenientes do *dataset* de validação (via protocolo UDP); um router central; e um servidor dedicado de análise (h2), atuando como nó *Edge* de Inteligência Artificial.

Nesta topologia, o tráfego injetado por h1 atravessa o router central, que está configurado para redirecionar o fluxo de rede para o servidor h2. É neste nó *Edge* que ocorre a interceção dos pacotes, a extração de *features* em tempo real e a submissão do tráfego à inferência do modelo treinado. Ao separar o encaminhamento básico (router) do processamento analítico (h2), este ambiente de *stress-test* permitiu simular não apenas a eficácia da classificação lógica do modelo, mas sobretudo as restrições físicas inerentes a uma rede em produção, tais como a latência imposta pelo redirecionamento, engarrafamentos de *buffer* e as inevitáveis perdas de pacotes.

Na Figura 5.1, é possível observar a arquitetura montada no Mininet.

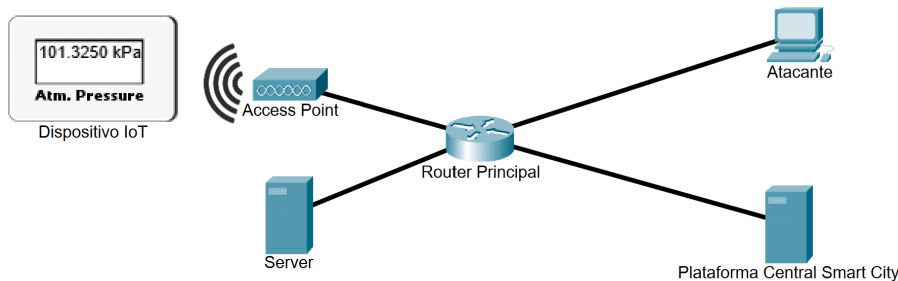


Figura 5.1: Arquitetura montada no Mininet.

5.6 Resultados e Otimização da Mitigação

5.6.1 Mecanismo de Filtragem Rápida via Blacklist Dinâmica

Apesar do baixo tempo de inferência alcançado pelo modelo MLP (~ 1 ms), submeter a totalidade do tráfego volumétrico de um ataque DDoS (milhões de pacotes) à análise da Inteligência Artificial resultaria no esgotamento prematuro do processador do *Edge Router*. Para contornar esta limitação arquite-

tural, foi implementado um sistema de mitigação híbrido com recurso a uma *Blacklist* Dinâmica.

O fluxo de funcionamento dita que apenas os pacotes iniciais de uma origem desconhecida são sujeitos à carga computacional da inferência por *Machine Learning*. No momento em que a rede neuronal sinaliza um IP de origem como malicioso, esse endereço é imediatamente adicionado a uma tabela de bloqueio (em memória *cache*). Consequentemente, os milhares de pacotes subsequentes desse atacante são descartados de forma instintiva (*drop*) nas camadas primárias da infraestrutura de rede lidas em tempo real, contornando o *pipeline* da IA. Esta simbiose entre inteligência preditiva e filtragem tradicional baseada em regras conferiu ao sistema uma escalabilidade formidável face a injeções de tráfego de alta densidade.

5.6.2 Métricas de Avaliação

Para quantificar o sucesso preditivo e a viabilidade operacional dos modelos em ambiente de *Edge Computing*, utilizou-se um conjunto de quatro métricas fundamentais:

- **Accuracy Global:** Representa a percentagem total de pacotes corretamente classificados pelo modelo. Embora intuitiva, pode ser enganosa em dados desbalanceados.
- **F1-Score Ponderado:** É a média harmónica entre a Precisão (falsos alarmes) e o *Recall* (capacidade de apanhar o *hacker*). Esta métrica é o indicador real de robustez, pois penaliza modelos que ignoram ataques raros.
- **Tempo Médio de Inferência (Latência):** Avalia o tempo, em milissegundos (ms), que o modelo demora a processar os dados de um pacote e a emitir uma decisão de bloqueio ou permissão. Numa arquitetura IoT, uma latência de processamento o mais próxima possível de 1 milissegundo é crítica para evitar o esgotamento do *buffer* do *router* e garantir a fluidez da rede.
- **Estabilidade da Latência (Variância / Jitter):** Analisada através de diagramas de dispersão estatística (*Boxplot*), avalia a consistência do tempo de resposta ao longo do teste. Modelos com alta variabilidade apresentam picos de latência imprevisíveis que, durante ataques volumétricos, resultariam no colapso do tráfego legítimo por incapacidade momentânea de processamento.

5.6.3 Análise do Desempenho Global

A Figura 5.2 apresenta a comparação das métricas globais entre os modelos avaliados.

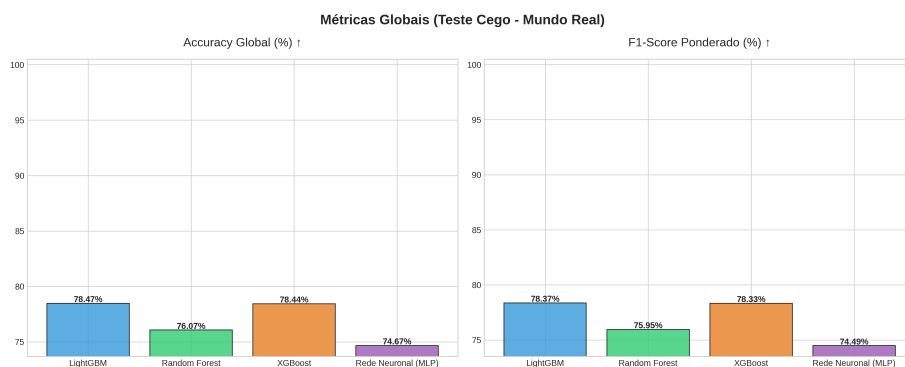


Figura 5.2: Métricas Globais no Teste Cego - Mundo Real.

Observando os resultados, constata-se que o LightGBM e o XGBoost mantiveram-se como os modelos com melhor equilíbrio global em termos de precisão técnica, apresentando valores de F1-Score muito similares.

Contudo, para um cenário de *Edge Computing*, a avaliação não se pode esgotar na precisão preditiva. A Figura 5.3 ilustra o tempo médio que cada modelo requer para realizar a inferência de um único pacote.

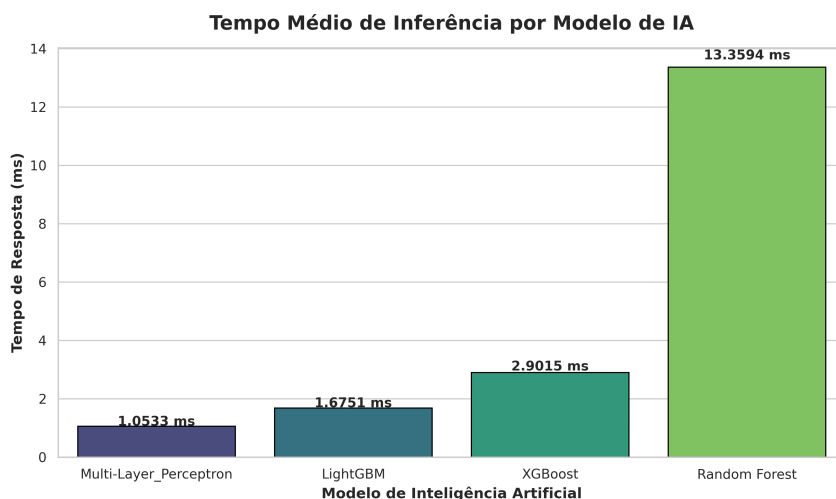


Figura 5.3: Tempo Médio de Inferência por Modelo de IA.

Os dados da latência alteram drasticamente a perspectiva de viabilidade. Modelos profundos como o Random Forest apresentam latências inviáveis (> 13 ms) para processamento em tempo real. Em contrapartida, a arquitetura matricial do Multi-Layer Perceptron (MLP) permitiu alcançar a maior velocidade operacional (pouco superior a 1 ms por pacote), revelando-se o algoritmo mais leve para atuar na periferia da rede. Esta eficiência computacional do MLP, comparada aos restantes, é reforçada pela análise de variância da latência visível na Figura 5.4.

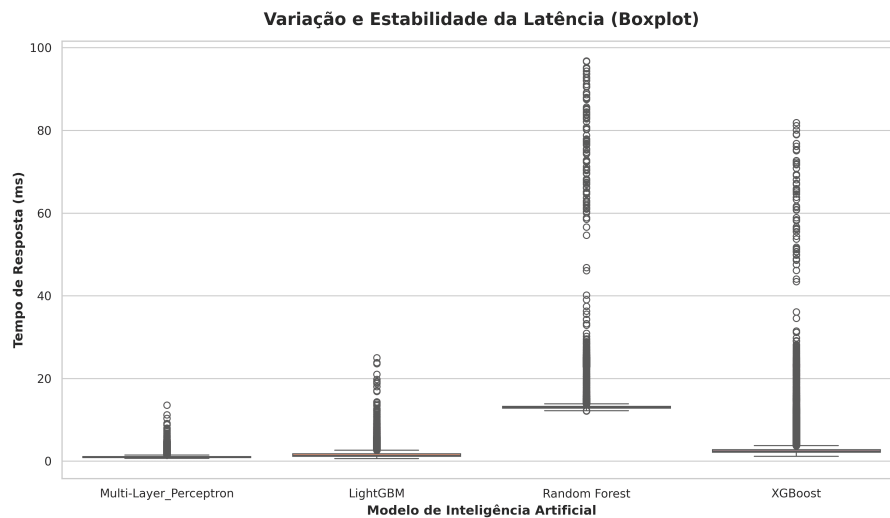


Figura 5.4: Variação e Estabilidade da Latência (Boxplot).

5.6.4 Recall por Anomalia

A análise da taxa de acertos por classe (Figura 5.5) revela o comportamento individual dos modelos perante cada tipo de ameaça e tráfego legítimo, constituindo o indicador mais granular da robustez do sistema em cenários reais.

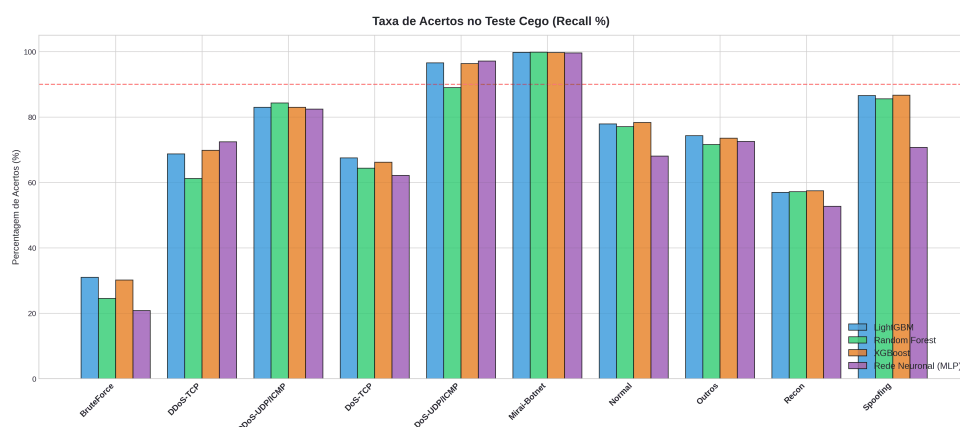


Figura 5.5: Taxa de Acertos (Recall %) por Tipo de Anomalia.

Ataques Volumétricos (DDoS/DoS TCP e UDP/ICMP): Os modelos evidenciaram desempenhos distintos consoante o protocolo. As variantes UDP e ICMP atingiram taxas de *Recall* entre 83% e 97%, beneficiando da natureza volumosa e estatisticamente distinta destes ataques. As variantes TCP apresentaram valores mais modestos, entre 62% e 72%, refletindo uma maior diversidade nos padrões de tráfego malicioso TCP sem necessidade de inspeção ao nível do *payload*.

Mirai Botnet e Spoofing: Estas classes registaram igualmente taxas de deteção elevadas, refletindo a agressividade dos padrões de tráfego gerados por botnets, que produzem anomalias estatísticas facilmente capturadas pela análise de fluxos.

Tráfego Normal: A generalidade dos modelos garantiu taxas de permissão do tráfego legítimo entre 68% e 79%, sendo crítico manter estes valores elevados para evitar interrupções de serviço causadas por falsos positivos numa infraestrutura de *Smart City*. O modelo MLP registou o valor mais conservador desta métrica (~68%), uma troca aceite face à sua superioridade na latência operacional.

É importante reconhecer que o valor de *Recall* do tráfego Normal registado pelo MLP (~68%) representa a principal limitação operacional do modelo eleito. Na prática, implica que aproximadamente 32% do tráfego legítimo pode ser incorretamente sinalizado como ameaça (*falso positivo*). Esta troca foi aceite deliberadamente em favor da estabilidade de latência, mas o seu impacto pode ser mitigado em trabalho futuro através do ajuste fino do limiar de decisão (*threshold*) do MLP, reduzindo a taxa de falsos positivos sem comprometer significativamente a deteção de ameaças reais.

BruteForce e Recon: Estas duas classes apresentaram as taxas de *Recall*

mais reduzidas entre todos os modelos avaliados. Esta limitação é inerente à metodologia de extração de *features* baseada em fluxos estatísticos adotada: ataques furtivos como o *Recon* ou sessões de força bruta geram volumes de tráfego reduzidos e padrões estatísticos subtis, que só seriam plenamente distinguíveis através da Inspeção Profunda de Pacotes (DPI) ao nível do *payload*. Contudo, a adoção de DPI foi deliberadamente excluída do âmbito deste projeto por incompatibilidade com os requisitos de baixa latência e privacidade de dados impostos pelo paradigma de *Edge Computing*, conforme discutido na Secção 2.3. Esta é uma limitação conhecida e aceite do sistema, cujo endereçamento constitui uma das principais linhas de trabalho futuro propostas no Capítulo 6.

5.7 Decisão Final e Conclusões

A seleção do algoritmo ideal para implementação num ambiente de *Edge Computing* não se pode basear unicamente na maximização isolada da taxa de acertos. Exige, pelo contrário, uma avaliação rigorosa do compromisso (*trade-off*) entre a eficácia preditiva e o custo computacional imposto ao *hardware* da rede.

Conforme ilustrado na Figura 5.6, os quatro modelos avaliados demonstraram níveis de exatidão global estatisticamente próximos, situados na faixa dos 74% a 78%.

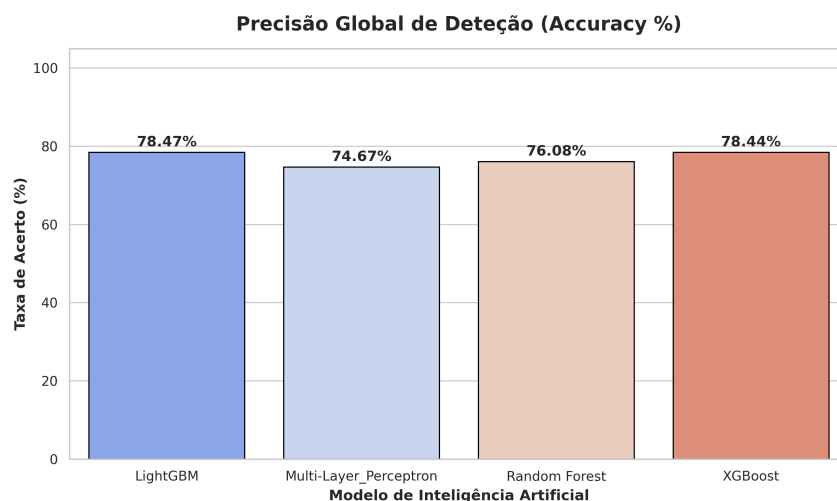


Figura 5.6: Precisão Global de Detecção (Accuracy %) por modelo.

No contexto das telecomunicações, mais crítico do que a latência média é a estabilidade desse tempo de resposta, vulgarmente conhecida por *jitter*. A análise do *Boxplot* na Figura 5.4 revela que os modelos baseados em árvores (*Random Forest* e *XGBoost*) sofrem de uma elevada variância, apresentando picos anómalos de latência (*outliers*) que chegam a atingir os 80 e 100 ms. Num *Edge Router* sob o efeito de um ataque volumétrico massivo (DDoS), esta instabilidade provocaria o rápido esgotamento do *buffer* de memória e o consequente bloqueio da rede (*Denial of Service* autoinduzido). O MLP, por sua vez, destaca-se por uma variância praticamente nula, garantindo um comportamento predizível e determinístico sob stress de rede.

Adicionalmente, a análise da variância individual de cada modelo, já ilustrada na Figura 5.4, confirma que o MLP apresenta uma caixa interquartílica extremamente compacta e um número reduzido de *outliers*, comparado com o *Random Forest* (picos até ~97 ms) e o *XGBoost* (picos até ~82 ms). Isto comprova que o tempo de processamento do MLP é não apenas rápido, mas predizível e determinista — uma propriedade fundamental para garantir a estabilidade do *Edge Router* sob ataques volumétricos de alta densidade.

Conclusão: Perante a conjugação destas métricas físicas e lógicas, o **Multi-Layer Perceptron (MLP)** é declarado o algoritmo vencedor e o motor do sistema de segurança deste projeto. Embora o MLP ceda uma margem de exatidão pura face ao *LightGBM* (74,67% vs. 78,47%), esta diferença de 3,8 pontos percentuais revela-se operacionalmente aceitável no contexto específico de *Edge Computing* sob ataque volumétrico. O argumento central reside não na magnitude da exatidão, mas no comportamento sob stress: enquanto o *Random Forest* e o *XGBoost* registam picos de latência de ~97 ms e ~82 ms respetivamente — suficientes para esgotar o *buffer* do *Edge Router* e provocar um *Denial of Service* autoinduzido — o MLP mantém uma variância praticamente nula com tempo de inferência inferior a 1 ms por pacote (cf. Figura 5.4). Em cenários de ataques de alta densidade, a previsibilidade determinística sobrepõe-se à precisão estatística marginal, conforme exigido pelo requisito **RNF01**. Aliado ao mecanismo de *Blacklist* Dinâmica, o modelo MLP confere ao sistema a escalabilidade perfeita para proteger ambientes de *Smart Cities*.

Conclusões e Trabalho Futuro

6.1 Conclusões Principais

A proliferação de dispositivos da *Internet of Things* (IoT) em ambientes críticos, como as *Smart Cities*, exige uma mudança de paradigma na cibersegurança. Este projeto propôs, desenvolveu e validou uma arquitetura de Prevenção de Intrusões (IPS) autónoma, deslocando a inteligência analítica da nuvem para a periferia da rede (*Edge Computing*).

A principal conclusão extraída deste trabalho é que a viabilidade de um sistema de segurança preditivo operando num *Edge Router* não depende exclusivamente da precisão estatística máxima, mas sim de um compromisso rigoroso entre a eficácia preditiva e a latência operacional. Ao avaliar empiricamente quatro algoritmos de *Machine Learning* distintos, provou-se que modelos baseados em árvores de decisão profundas (como o *Random Forest*) geram latências e flutuações de processamento incompatíveis com o encaminhamento de tráfego em tempo real.

Em contrapartida, o modelo *Multi-Layer Perceptron* (MLP) revelou-se a solução ótima para este cenário. A sua arquitetura matricial permitiu classificar anomalias complexas com um tempo de inferência ultrarrápido (próximo de 1 milissegundo por pacote) e com uma variância praticamente nula.

Adicionalmente, a implementação de uma topologia de emulação realista (Mininet) demonstrou o sucesso da abordagem híbrida do sistema. A conjugação da inteligência do MLP com o bloqueio a nível de rede (através de uma *Blacklist* dinâmica) provou ser capaz de mitigar injeções massivas de tráfego malicioso, como ataques de Negação de Serviço (DDoS) e *Botnets* (Mirai). O sistema descartou eficazmente as ameaças nas primeiras camadas da rede, preservando o poder computacional do *hardware* e garantindo o funciona-

mento ininterrupto dos dispositivos legítimos da cidade.

6.2 Trabalho Futuro

Apesar dos resultados promissores alcançados, a segurança em redes IoT é um campo em constante mutação. A arquitetura desenvolvida abre portas a diversas linhas de investigação futura para robustecer o sistema.

Em primeiro lugar, a abordagem de extração de *features* baseada em fluxos estatísticos, embora fundamental para garantir a baixa latência e a privacidade dos dados, limita a detecção de anomalias ao nível da aplicação (Camada 7) ou ataques altamente furtivos. Trabalhos futuros poderão explorar a integração de Inspeção Profunda de Pacotes (DPI) de forma seletiva, em que o *payload* só é analisado caso o modelo MLP detete um comportamento estatisticamente suspeito, criando assim um mecanismo de verificação em duas etapas.

Em segundo lugar, seria de elevado interesse integrar este nó *Edge* com controladores de redes definidas por *software* (*Software-Defined Networking* - SDN). Num cenário de *Smart City*, se um nó intercetar um atacante *Zero-Day*, o controlador SDN poderia propagar a *Blacklist* gerada a todos os restantes *routers* da cidade instantaneamente, criando um sistema imunológico distribuído e automatizado.

Por fim, a expansão desta arquitetura poderia beneficiar da adoção de algoritmos de *Federated Learning* (Aprendizagem Federada). Esta tecnologia permitiria que múltiplos servidores *Edge* espalhados por diferentes cidades treinassem e melhorassem os seus modelos de Inteligência Artificial de forma colaborativa, partilhando apenas as atualizações matemáticas do modelo, sem nunca transferir o tráfego sensível dos cidadãos para um servidor central, elevando assim os padrões de privacidade e segurança da infraestrutura ciberfísica.

Bibliografia

- [1] Shad Roundy, Paul K. Wright, and Jan M. Rabaey. *Energy Scavenging for Wireless Sensor Networks with Special Focus on Vibrations*. ISBN: 978-1-4020-7663-3. Springer, 2003.
- [2] H. Tschofenig and E. Baccelli. Cyberphysical security for the masses: A survey of the internet protocol suite for internet of things security. *IEEE Security Privacy*, 17(5):47–57, 2019.
- [3] IEEE. Técnicas de detecção de anomalias em iot e desafios das mesmas. *IEEE Xplore*, 2021. [Online] <https://ieeexplore.ieee.org/document/9533252>.
- [4] Mohammed Ali Al-Garadi, Amr Mohamed, Abdulla Khalid Al-Ali, Xiaojiang Du, Ihsan Ali, and Mohsen Guizani. A survey of machine and deep learning methods for internet of things (iot) security. *IEEE Communications Surveys & Tutorials*, 22(3):1646–1685, 2020. [Online] <https://ieeexplore.ieee.org/document/8977365>.
- [5] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [6] Daniel S. Berman, Anna L. Buczak, Jeffrey S. Chavis, and Cynthia L. Corbett. A survey of deep learning methods for cyber security. *Information*, 10(4):122, 2019. [Online] <https://www.mdpi.com/2078-2489/10/4/122>.
- [7] Canadian Institute for Cybersecurity. Cic iot dataset 2023. [Online], 2023. <https://www.unb.ca/cic/datasets/iotdataset-2023.html>.
- [8] Jiasi Chen and Xukan Ran. Deep learning with edge computing: A review. *Proceedings of the IEEE*, 107(8):1655–1674, 2019. [Online] <https://ieeexplore.ieee.org/document/8736011>.
- [9] Sahil Garg, Kuljeet Kaur, Neeraj Kumar, Georges Kaddoum, Sherali Zedally, and Mohammad Mehedi Hassan. A novel sdn-enabled iot anomaly detection using ensemble learning. *IEEE Access*, 2020. [Online] <https://pmc.ncbi.nlm.nih.gov/articles/PMC7256579/>.

- [10] Wes McKinney. Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010.
- [11] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [12] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- [13] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [14] Monika Roopak, Gui Yun Tian, and Jonathon Chambers. Deep learning models for cyber security in iot networks. In *2019 IEEE 9th Annual Computing and Communication Workshop and Conference (CCWC)*, pages 0452–0457, 2019.
- [15] Iftikhar Ahmad, Mohammed Basher, Muhammad Javed Iqbal, and Amin Rahim. Performance comparison of support vector machine, random forest, and extreme learning machine for intrusion detection. *IEEE Access*, 6:33789–33795, 2018.